Programmation avancée 2025 — Université Paris Cité

Simulateur de trafic urbain — Paris 13e

Optimisation des feux par apprentissage par renforcement

GBAGUIDI Nerval

LENGRAI Jordan

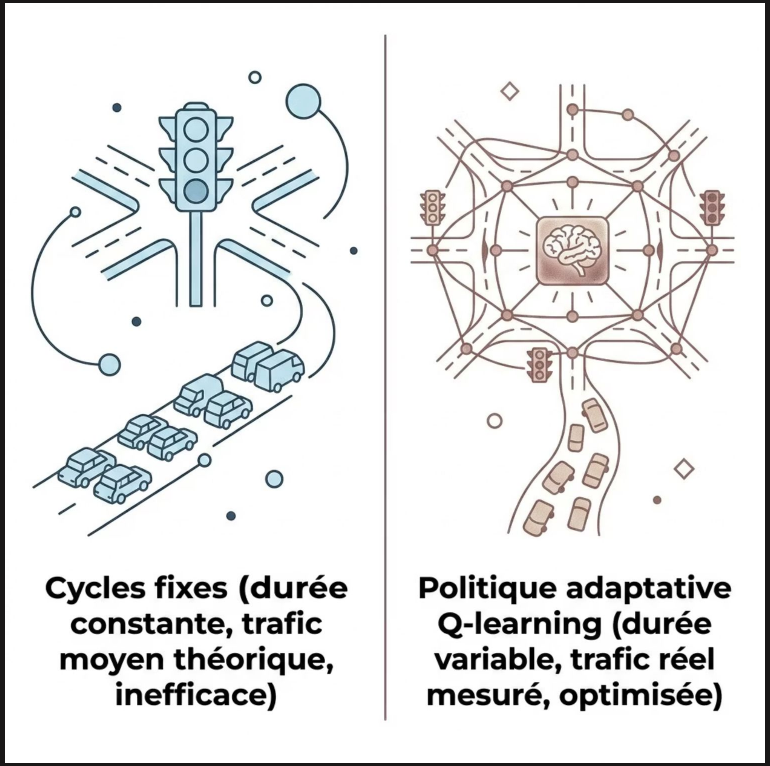
PROBLÉMATIQUE

Le problème et notre objectif

Dans une ville, des voitures attendent au feu rouge alors que la rue d'en face est vide. Les feux suivent des cycles fixes, calculés pour un trafic moyen qui n'existe jamais.

> Notre objectif : construire un outil d'aide à la décision pour optimiser un réseau de feux existant. Sans toucher aux routes. Sans dépenser un centime en travaux. Juste en changeant la durée des phases.

On part d'un quartier réel — le **13e arrondissement de Paris** — et on mesure le gain qu'un agent adaptatif peut apporter par rapport à des cycles fixes.



FONCTIONNALITÉS

Ce que notre produit fait

À partir d'un quartier réel (données OpenStreetMap), le logiciel produit trois résultats principaux :

1

Simulation complète

Circulation avec itinéraires Dijkstra, files d'attente, priorités, suivi entre véhicules et sens uniques.

2

Comparaison chiffrée

Politique de feux actuelle (cycles fixes) versus politique adaptative pilotée par un agent Q-learning.

3

Recommandations exportées

Fichier `recommended_cycles.json` : durée de phase recommandée, gain attendu et verdict par carrefour.

Trois scénarios d'utilisation

Mairie — tester une stratégie de feux avant déploiement

Bureau d'études — identifier les carrefours critiques d'un quartier

Chercheur — quantifier le gain d'une politique adaptative

ARCHITECTURE

Vue d'ensemble du logiciel

Quatre paquets principaux (`python/`)

`simulation/`

Moteur de trafic : graphe routier, véhicules, feux, métriques

`gui/`

Interface visuelle Pygame

`rl/`

Apprentissage par renforcement : agent, environnement, entraînement, benchmark, analyse

`data_tools/`

Récupération et nettoyage des données OpenStreetMap

Données et sorties

Entrée : `data/map_clean.json` — 7 969 nœuds routiers, 8 458 arcs, 326 feux dans 250 carrefours

Sorties : `results/q_table.json`, `results/benchmark.json`, `results/recommended_cycles.json`

Dépendances internes

- `gui` et `rl` dépendent tous deux de `simulation`
- `simulation` est autonome et testable sans GUI ni RL

MODULES

Décomposition des modules

Trafic

- `graph.py` : graphe routier, plus court chemin Dijkstra
- `vehicle_motion.py` : mouvement des véhicules
- `vehicle_queues.py` : gestion des files
- `vehicle_intersections.py` : résolution des conflits aux intersections
- `lights.py` : feux, groupes de carrefours, contraintes physiques de sécurité

Visualisation

- `app.py` : boucle principale Pygame
- `renderer.py` : affichage carte, véhicules, feux et HUD

Apprentissage par renforcement

- `env.py` : environnement RL avec API reset / step / observation / reward
- `agent.py` : agent Q-learning (alpha, gamma, epsilon-greedy, Q-table)
- `train.py` : entraînement multi-seed avec sauvegarde du meilleur snapshot
- `benchmark.py` : comparaison baseline contre RL, moyenne sur plusieurs seeds
- `analysis.py` : recommandations de cycles par carrefour

CONCEPTION

Choix de conception et modularité

Pourquoi cette organisation ?

Séparation simulation / RL — La simulation tourne sans agent, l'agent peut s'entraîner sans GUI. Tests isolés possibles.

Q-table partagée — Un dictionnaire Python pour les 250 carrefours. Ce qui est appris à un carrefour sert aux autres. Apprentissage rapide et transférable.

RLController modulaire — Se branche sur la boucle Pygame sans rien modifier dans `simulation/`.

Évolutivité — Changer le quartier, c'est changer `map_clean.json`. Tout le pipeline fonctionne sur n'importe quelle zone OSM.

Robustesse : Le `GroupSignalRuntime` garde les contraintes physiques (vert minimum 6 s, jaune obligatoire, phase tout rouge entre deux verts). Même si l'agent demande l'impossible, le simulateur refuse.

Répartition du travail

Organisation : développement par couches successives sur le semestre. D'abord la simulation seule, puis l'interface visuelle, enfin la couche RL et le benchmark. Tests unitaires ajoutés en continu sur la partie RL.

Jordan

- Récupération et nettoyage des données OSM (`data_tools/`)
- Interface Pygame (`app.py`, `renderer.py`)
- Couche apprentissage par renforcement complète (agent Q-learning, environnement, entraînement multi-seed, benchmark, analyse)

Nerval

- Métriques de trafic en temps réel (`simulation/metrics.py`)
- Gestion des véhicules : mouvement, files, intersections, géométrie (`simulation/vehicle*.py`)
- Feux de circulation : cycles, groupes de carrefours, contraintes de sécurité des phases (`simulation/lights.py`)

RÉSULTATS

Difficultés, tests et résultats mesurés

Difficultés côté apprentissage

Explosion d'états

Un état à 8 dimensions donnait 61 440 combinaisons, trop pour 80 épisodes. **Solution :** ramener à 4 dimensions, soit 240 états.

Biais de sélection

L'agent débloquait des trajets longs qui masquaient le gain réel. **Solution :** ajout d'indicateurs robustes (médiane, vitesse moyenne effective, attente aux feux).

Variabilité des scénarios

Un seul scénario peut être chanceux. **Solution :** benchmark multi-seed (42 à 46), métriques moyennées sur 5 simulations.

18 tests unitaires valident l'agent, l'environnement et le verdict du benchmark.

Résultats mesurés

300 s simulées, 100 véhicules cibles, moyenne sur 5 seeds

+40,9%

Débit

Cible +10% — PASS

+25,6%

Vitesse effective

Cible +10% — PASS

-14,4%

Attente aux feux

Réduction de l'attente moyenne

-19,6%

File moyenne

Réduction de la file d'attente

Sur les 10 carrefours les plus engorgés, 8 sont améliorés — jusqu'à **-84 %** de file moyenne sur le carrefour le plus chargé.

RÉSULTATS

Difficultés, tests et résultats mesurés

Difficultés côté simulation

Circulation réaliste

Modèle de suivi (vehicle_motion.py) calculant à chaque pas la vitesse cible en fonction de l'espace libre devant.

Priorités aux intersections

Résolveur de conflits par intersection (vehicle_intersections.py) qui sérialise les passages.

Files d'attente

Module dédié (vehicle_queues.py) maintenant une file par feu, libérant les véhicules au passage au vert.

CODE

L'agent Q-learning – `rl/agent.py`

Tout l'agent Q-learning tient en 50 lignes, sans PyTorch et sans Gym. C'est le cœur du projet.

Trois choix techniques

1 Q-table comme dictionnaire Python

`{ état : [Q_KEEP, Q_SWITCH, Q_EXTEND] }` — Une seule table sert aux 250 carrefours, rendant l'apprentissage transférable.

2 Initialisation paresseuse

Le helper `_q()` initialise l'entrée seulement quand l'état est observé pour la première fois. Pas de pré-remplissage de 240 entrées inutiles.

3 Équation de Bellman

La mise à jour Q-learning tient en une ligne lisible. Sauvegarde et rechargement en JSON : la Q-table est portable et inspectable à la main.

```
class QLearningAgent:

    def __init__(self, alpha=0.1, gamma=0.9, epsilon=1.0,
                 epsilon_min=0.05, epsilon_decay=0.95):
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon
        self.Q = {}          # { etat : [Q_KEEP, Q_SWITCH, Q_EXTEND] }
        self.rng = random.Random(0)

    def choose(self, state):
        if self.rng.random() < self.epsilon:
            return self.rng.randrange(NB_ACTIONS)
        return self.greedy_action(state)

    def update(self, state, action, reward, next_state):
        current  = self._q(state)[action]
        next_max = max(self._q(next_state))
        new_value = current + self.alpha * (
            reward + self.gamma * next_max - current
        )
        self._q(state)[action] = new_value

    def _q(self, state):
        if state not in self.Q:
            self.Q[state] = [0.0] * NB_ACTIONS
        return self.Q[state]
```

CONCLUSION

Version 2 et leçons apprises

À quoi ressemblerait la V2

01

Coordonner les carrefours voisins entre eux (vague verte sur un axe)

02

Enrichir l'état avec la saturation $\rho = \lambda / \mu$ par phase, issue de la théorie des files d'attente

03

Entraînement plus long pour couvrir l'intégralité des 240 états

04

Remplacer la cible officielle « temps moyen de trajet » par le temps médian, plus robuste au biais de sélection

Ce que nous changerions

Démarrer avec un état discret plus simple : nous avons perdu du temps avec un état trop riche qui empêchait l'apprentissage

Mettre en place le benchmark multi-seed dès le départ

Écrire les indicateurs robustes avant les cibles officielles, pour éviter de courir après une métrique trompeuse

---

> Nous avons montré qu'il est possible d'améliorer mesurablement le trafic d'un quartier réel sans toucher aux routes, juste en changeant la durée des phases — et l'outil produit, carrefour par carrefour, la recommandation à appliquer.

Merci de votre attention.